

ANSH THAPA

B.TECH CSE

CU24250224

Experiment No. 6

Implementation and Comparative Analysis of Image Segmentation Techniques using Python and OpenCV

Aim

To implement and evaluate various image segmentation techniques for partitioning digital images into meaningful regions, thereby facilitating object localization, scene understanding, and subsequent computer vision tasks

Purpose

Image segmentation is one of the most important stages in computer vision, where an image is divided into multiple meaningful regions or objects based on pixel intensity, color, texture, or boundaries. Effective segmentation simplifies image analysis by isolating the regions of interest (ROI), making tasks such as object detection, medical diagnosis, autonomous navigation, and industrial inspection more accurate and efficient.

In this experiment, students will implement multiple image segmentation techniques, including Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation, and KMeans Clustering, and compare their performance on real-world images

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, Matplotlib, and Scikit-learn
- Understanding of grayscale images, image histograms, and clustering concepts
- Jupyter Notebook, Google Colab, or Visual Studio Code

```
In [3]: import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Load image
image = cv2.imread("/content/Screenshot 2026-08-28 004515.png")

if image is None:
    print("Image not found")
else:
    print("Image loaded successfully")

# Convert BGR to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

Image loaded successfully

Original Image



```
In [4]: # Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Gaussian Blur
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred")
plt.axis("off")

plt.show()
```

Grayscale



Gaussian Blurred



```
In [5]: _, global_thresh = cv2.threshold(
        blurred, 127, 255, cv2.THRESH_BINARY
    )

plt.imshow(global_thresh, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()
```

Global Thresholding



```
In [6]: otsu_value, otsu_thresh = cv2.threshold(
        blurred, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

print("Otsu Threshold Value:", otsu_value)

plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()
```

Otsu Threshold Value: 105.0

Otsu's Thresholding



```
In [7]: adaptive_thresh = cv2.adaptiveThreshold(
        blurred,
        255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        11,
        2
    )

plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Thresholding")
```

```
plt.axis("off")
plt.show()
```

Adaptive Thresholding



```
In [8]: # Use Otsu result
binary = otsu_thresh.copy()

# Remove noise using morphological opening
kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(
    binary, cv2.MORPH_OPEN, kernel, iterations=2
)

# Find sure background
sure_bg = cv2.dilate(opening, kernel, iterations=3)

# Distance transform
dist_transform = cv2.distanceTransform(
    opening, cv2.DIST_L2, 5
)

# Find sure foreground
_, sure_fg = cv2.threshold(
    dist_transform,
    0.5 * dist_transform.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

# Marker labelling
_, markers = cv2.connectedComponents(sure_fg)

markers = markers + 1
markers[unknown == 255] = 0

# Apply watershed
watershed_image = image.copy()

markers = cv2.watershed(
    watershed_image,
    markers
)

# Mark boundaries
watershed_image[markers == -1] = [0, 0, 255]

plt.figure(figsize=(10, 5))
plt.imshow(cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB))
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()
```

Watershed Segmentation



```
In [9]: # Convert image to RGB
rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Reshape pixels
pixels = rgb.reshape((-1, 3))
pixels = np.float32(pixels)

# Number of clusters
k = 3

# Apply K-Means
kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixels)

# Get cluster centers
centers = np.uint8(kmeans.cluster_centers_)

# Reconstruct segmented image
segmented = centers[labels]
segmented = segmented.reshape(rgb.shape)

plt.imshow(segmented)
plt.title("K-Means Color Segmentation")
plt.axis("off")
plt.show()
```

K-Means Color Segmentation



Technique	Main Idea	Accuracy	Speed	Best Use
-----------	-----------	----------	-------	----------

Global Thresholding	Fixed threshold	Good for simple images	Fast	Simple foreground/background
Otsu	Automatic threshold	Better than fixed threshold in many cases	Fast	Images with clear intensity separation
Adaptive Thresholding	Local threshold	Good with uneven lighting	Moderate	Documents and uneven illumination
Watershed	Boundary-based segmentation	Good for touching objects	Moderate	Separating overlapping objects
K-Means	Color-based clustering	Depends on image/colors	Moderate	Color segmentation

```
In [10]: plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_thresh, cmap="gray")
plt.title("Global Threshold")
plt.axis("off")

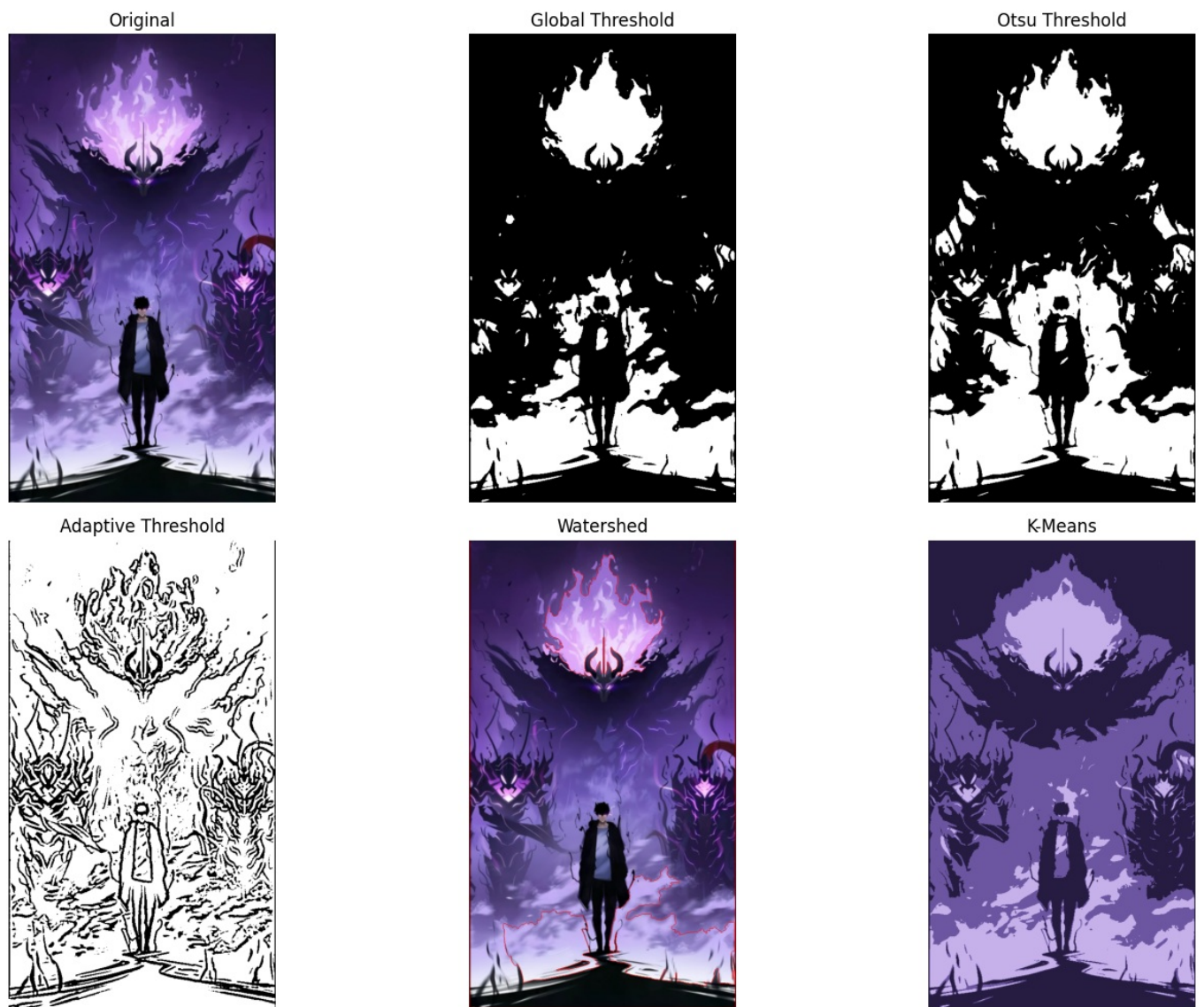
plt.subplot(2, 3, 3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu Threshold")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Threshold")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB))
plt.title("Watershed")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Observation

Global Thresholding: Simple and fast but depends on the selected threshold.

Otsu: Automatically finds a suitable threshold.

Adaptive Thresholding: Better when illumination is not uniform.

Watershed: Useful for separating touching objects.

K-Means: Useful when objects have different colors.

10. Conclusion

Image segmentation divides an image into meaningful regions or objects. Global and Otsu thresholding are useful for simple foreground-background separation. Adaptive thresholding performs better with uneven lighting. Watershed is useful for separating touching objects, while K-Means is effective for color-based segmentation.

Image segmentation is widely used in medical imaging, satellite image analysis, autonomous vehicles, object detection, face analysis, and other computer vision applications.

Questions

1. What is image segmentation? Why is it important in computer vision?

Answer: Image segmentation is the process of dividing an image into different meaningful regions or objects. It helps separate the foreground from the background or identify different objects in an image.

It is important because it helps a computer analyze, detect, and recognize objects easily. It is used in medical imaging, autonomous vehicles, satellite images, and object detection.

2. Differentiate between Image Segmentation and Image Classification.

Answer:

Image Segmentation	Image Classification
Divides an image into different regions or objects.	Assigns a label to the whole image.
Works at pixel or region level.	Works at image level.
Identifies where an object is present.	Identifies what the image contains.
Example: Separating a tumor from healthy tissue.	Example: Classifying an image as normal or abnormal.

3. Explain Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Answer:

Global Thresholding: It uses one fixed threshold value for the entire image to separate the foreground from the background.

Otsu's Thresholding: It automatically calculates the best threshold value by analyzing the intensity values of the image.

Adaptive Thresholding: It uses different threshold values for different regions of an image. It is useful when the image has uneven lighting.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Answer: The Watershed Algorithm is an image segmentation technique used to divide an image into different regions. It treats the image like a topographic surface and finds boundaries between objects.

It is useful for separating touching or overlapping objects. For example, it can separate two cells that are touching each other in a medical image.

5. How is K-Means Clustering applied to image segmentation?

Answer: K-Means clustering groups image pixels into a fixed number of clusters based on their color or intensity similarity.

The main steps are:

1. Select the number of clusters (K).
2. Assign pixels to the nearest cluster.
3. Calculate new cluster centers.
4. Repeat the process until the clusters become stable.

K-Means is commonly used for color-based image segmentation.

6. What are the advantages and limitations of traditional image segmentation techniques?

Answer:

Advantages:

- Simple and easy to implement.
- Fast execution.
- Usually does not require training data.
- Requires less computational power.

Limitations:

- May not work well with complex images.
 - Sensitive to noise and lighting changes.
 - Difficult to handle overlapping objects.
 - Accuracy depends on selected parameters.
-

7. What is the difference between Global Thresholding and Adaptive Thresholding?

Answer: Global Thresholding uses one threshold value for the complete image.

Adaptive Thresholding calculates different threshold values for different regions of the image.

Therefore, Global Thresholding works well for images with uniform lighting, while Adaptive Thresholding is better for images with uneven illumination.

8. Which segmentation technique is best for images with uneven illumination? Why?

Answer: Adaptive Thresholding is best for images with uneven illumination.

It calculates the threshold separately for different local regions. Therefore, even if some parts of the image are brighter or darker than others, the objects can still be segmented properly.

9. What is the role of image segmentation in advanced computer vision applications?

Answer: Image segmentation helps computer vision systems identify and separate important objects or regions from an image.

It is used in:

- Medical imaging for detecting tumors and organs.
 - Autonomous vehicles for identifying roads, vehicles, and pedestrians.
 - Satellite imaging for identifying land, water, and buildings.
 - Object detection for separating objects from the background.
 - Face analysis for identifying different regions of a face.
-

10. Compare traditional image segmentation techniques with deep learning-based segmentation methods such as U-Net and Mask R-CNN.

Answer: Traditional techniques such as Global Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed, and K-Means use predefined rules based on intensity, edges, or color. They are simple, fast, and usually do not require training data. However, their performance can decrease with complex images.

U-Net and Mask R-CNN are deep learning-based segmentation methods. They learn useful features automatically from labeled training images.

U-Net: It is mainly used for semantic segmentation, where pixels are assigned to different classes. It is widely used in medical image segmentation.

Mask R-CNN: It performs instance segmentation, where each individual object gets its own segmentation mask. It can detect and separate multiple objects of the same class.

Feature	Traditional Methods	U-Net	Mask R-CNN
Approach	Rule-based	Deep learning	Deep learning
Training data	Usually not required	Required	Required
Complexity	Low	High	High
Accuracy on complex images	Lower	High	High
Main purpose	Basic segmentation	Semantic segmentation	Instance segmentation
Examples	Otsu, K-Means, Watershed	Tumor segmentation	Object detection and segmentation

Conclusion: Traditional methods are suitable for simple images and low-computation tasks, while U-Net and Mask R-CNN are better for complex real-world segmentation. However, deep learning methods require labeled data, more computational power, and training time.